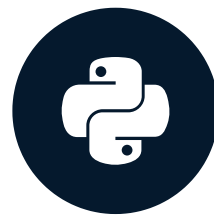


Ámbitos y funciones definidas por el usuario

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON



Hugo Bowne-Anderson
Instructor

Resumen exprés sobre ámbitos en funciones

- No todos los objetos son accesibles en todo el script
- Ámbito: parte del programa donde un objeto o nombre es accesible
 - Ámbito global: definido en el cuerpo principal de un script
 - Ámbito local: definido dentro de una función
 - Ámbito integrado: nombres del módulo predefinido built-ins

Ámbito global vs. local (1)

```
def square(value):  
    """Returns the square of a number."""  
    new_val = value ** 2  
    return new_val  
  
square(3)
```

9

new_val

```
<hr />-----  
NameError                                Traceback (most recent call last)  
<ipython-input-3-3cc6c6de5c5c> in <module>()  
<hr />-> 1 new_value  
NameError: name 'new_val' is not defined
```

Ámbito global vs. local (2)

```
new_val = 10

def square(value):
    """Returns the square of a number."""
    new_val = value ** 2
    return new_val

square(3)
```

9

new_val

10

Ámbito global vs. local (3)

```
new_val = 10

def square(value):
    """Returns the square of a number."""
    new_value2 = new_val ** 2
    return new_value2

square(3)
```

100

```
new_val = 20

square(new_val)
```

400

Ámbito global vs. local (4)

```
new_val = 10

def square(value):
    """Returns the square of a number."""
    global new_val
    new_val = new_val ** 2
    return new_val

square(3)
```

100

new_val

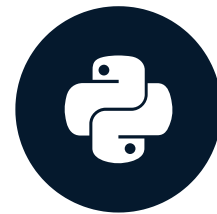
100

¡Vamos a practicar!

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON

Funciones anidadas

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON



Hugo Bowne-Anderson
Instructor

Funciones anidadas (1)

```
def outer( ... ):
    """ ... """
    x = ...

    def inner( ... ):
        """ ... """
        y = x ** 2
    return ...
```

Funciones anidadas (2)

```
def mod2plus5(x1, x2, x3):  
    """Devuelve el resto más 5 de tres valores."""  
  
    new_x1 = x1 % 2 + 5  
    new_x2 = x2 % 2 + 5  
    new_x3 = x3 % 2 + 5  
  
    return (new_x1, new_x2, new_x3)
```

Funciones anidadas (3)

```
def mod2plus5(x1, x2, x3):  
    """Devuelve el resto más 5 de tres valores."""  
  
    def inner(x):  
        """Devuelve el resto más 5 de un valor."""  
        return x % 2 + 5  
  
    return (inner(x1), inner(x2), inner(x3))
```

```
print(mod2plus5(1, 2, 3))
```

```
(6, 5, 6)
```

Devolver funciones

```
def raise_val(n):  
    """Devuelve la función interna."""  
  
    def inner(x):  
        """Eleva x a la potencia n."""  
        raised = x ** n  
        return raised  
  
    return inner
```

```
square = raise_val(2)  
cube = raise_val(3)  
print(square(2), cube(4))
```

4 64

Uso de nonlocal

```
def outer():  
    """Imprime el valor de n."""  
    n = 1  
  
    def inner():  
        nonlocal n  
        n = 2  
        print(n)  
  
    inner()  
    print(n)
```

```
outer()
```

```
2
```

```
2
```

Ámbitos de búsqueda

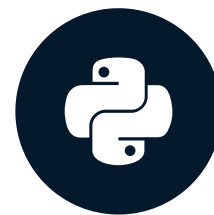
- Ámbito local
- Funciones envolventes
- Global
- Integrado

¡Vamos a practicar!

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON

Reunirlo todo

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON



Hugo Bowne-Anderson
Instructor

Próximos ejercicios:

- Funciones generalizadas:
 - Contar ocurrencias de cualquier columna
 - Contar ocurrencias de un número arbitrario de columnas

Añade un argumento por defecto

```
def power(number, pow=1):  
    """Raise number to the power of pow."""  
    new_value = number ** pow  
    return new_value
```

```
power(9, 2)
```

```
81
```

```
power(9)
```

```
9
```

Argumentos flexibles: *args (1)

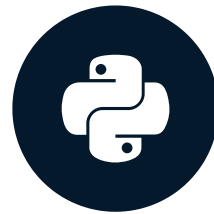
```
def add_all(*args):  
    """Sum all values in *args together."""  
  
    # Initialize sum  
    sum_all = 0  
  
    # Accumulate the sum  
    for num in args:  
        sum_all = sum_all + num  
  
    return sum_all
```

¡Vamos a practicar!

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON

Argumentos por defecto y flexibles

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON



Hugo Bowne-Anderson
Instructor

Aprenderás a:

- Escribir funciones con argumentos por defecto
- Usar argumentos flexibles
 - Pasar cualquier número de argumentos a una función

Añade un argumento por defecto

```
def power(number, pow=1):  
    """Eleva number a la potencia pow."""  
    new_value = number ** pow  
    return new_value
```

```
power(9, 2)
```

```
81
```

```
power(9, 1)
```

```
9
```

```
power(9)
```

```
9
```


Argumentos flexibles: *args (1)

```
def add_all(*args):  
    """Suma todos los valores de *args."""  
  
    # Inicializa la suma  
    sum_all = 0  
  
    # Acumula la suma  
    for num in args:  
        sum_all += num  
  
    return sum_all
```


Argumentos flexibles: *args (2)

```
add_all(1)
```

1

```
add_all(1, 2)
```

3

```
add_all(5, 10, 15, 20)
```

50

Argumentos flexibles: **kwargs

```
print_all(name="Hugo Bowne-Anderson", employer="DataCamp")
```

```
name: Hugo Bowne-Anderson  
employer: DataCamp
```

Argumentos flexibles: **kwargs

```
def print_all(**kwargs):  
    """Imprime los pares clave-valor en **kwargs."""  
  
    # Imprime los pares clave-valor  
    for key, value in kwargs.items():  
        print(key + ": " + value)
```

```
print_all(name="dumbledore", job="headmaster")
```

```
job: headmaster  
name: dumbledore
```

¡Vamos a practicar!

INTRODUCCIÓN A LAS FUNCIONES EN PYTHON